"Hello Everyone!"

Recursion

Note : Focus is on understanding Recursion and NOT optimization.

Recursion : → function calling itself

↳ solve answer of current problem using sub problem.

(Q) find sum of first N natural numbers.

↳ $\frac{N*(N+1)}{2}$ ✓

Recursion

$sum(5) = 1+2+3+4+5$

$= sum(4) + 5$

$sum(5) = sum(4) + 5$

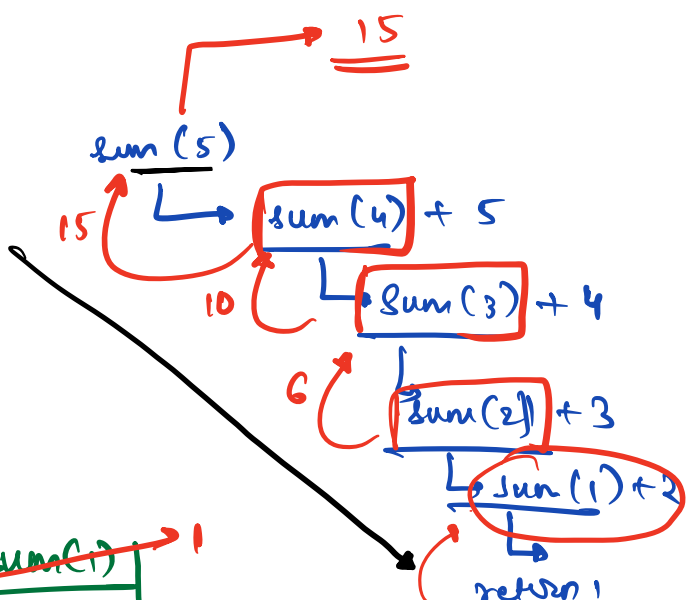
$sum(N) = sum(N-1) + N$

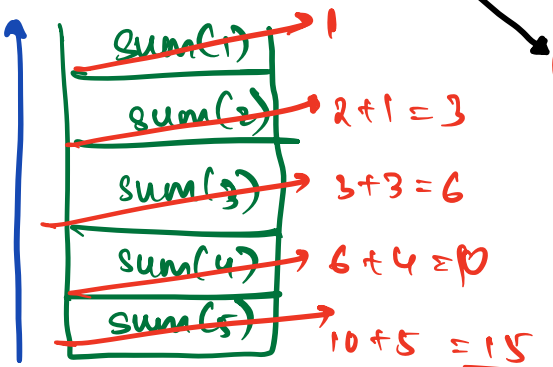
Steps to write code :-

1. Define what the function do.
2. Build logic on how to use subproblems to solve current problem.
3. Base condition
   - ↳ smallest subproblem for which we know the answer.

$sum(1) = 1$

```
int sum(N)
{
    if(N == 1)
        return 1
    return sum(N-1) + N
}
```





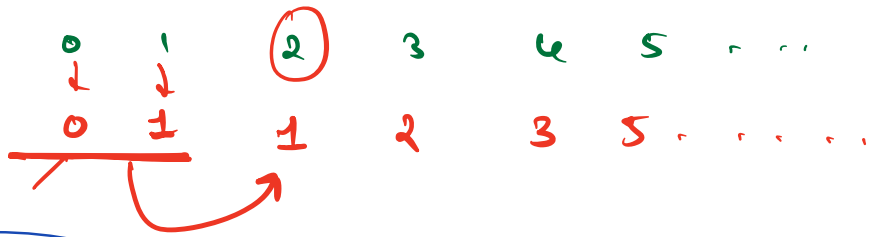
TC: O(N)

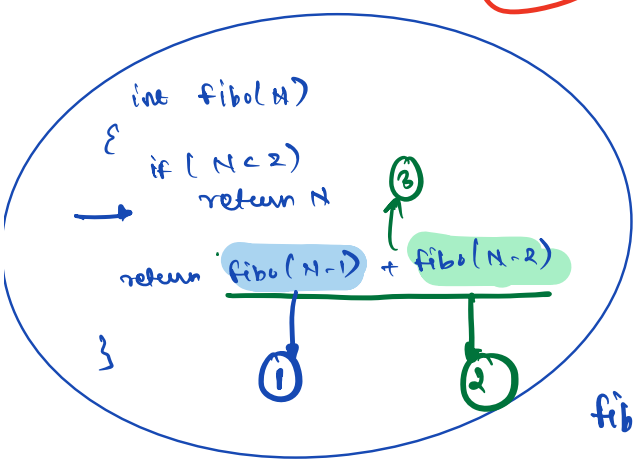
SC: O(N)

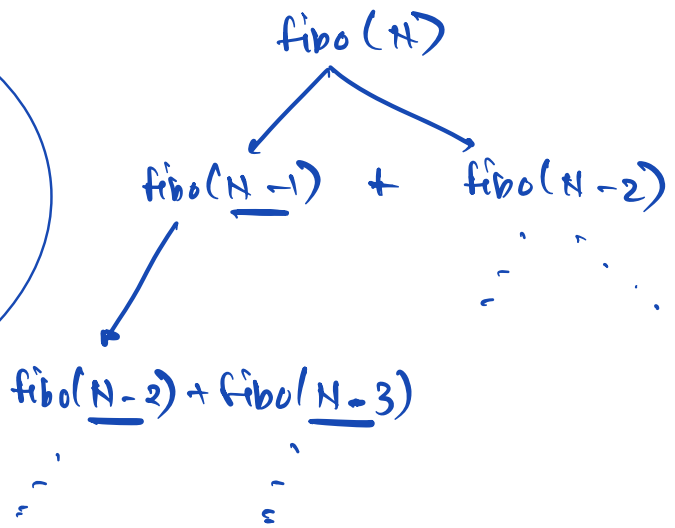
① Fibonacci Numbers

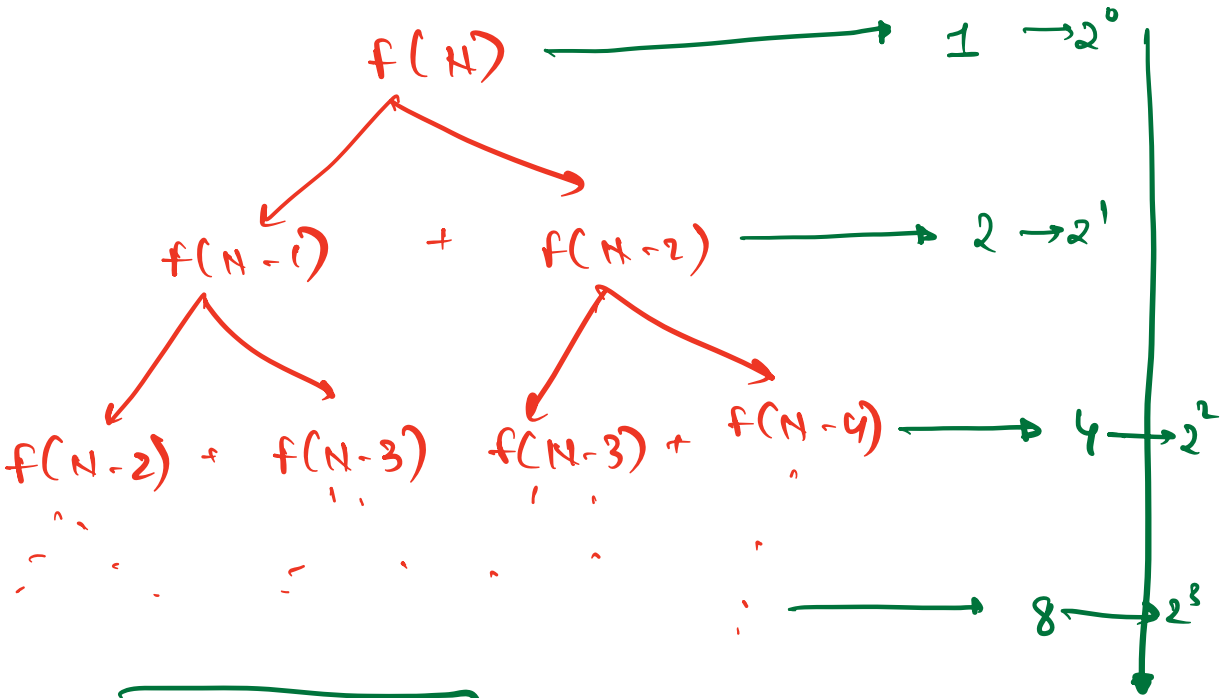
$f[i] = f[i-1] + f[i-2]$, $i >= 2$

$f[i] = i$, $i < 2$, $i >= 0$



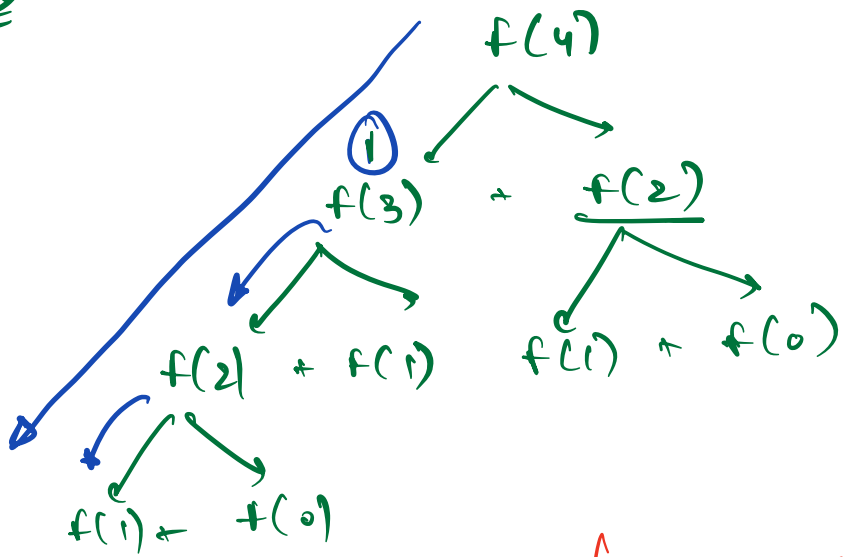


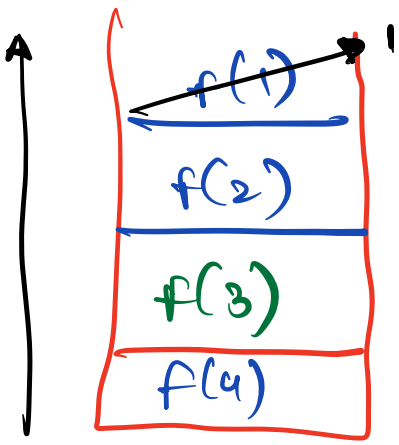




TC: $O(2^N)$

(4)





SC: O(N)

Output Questions :-

①

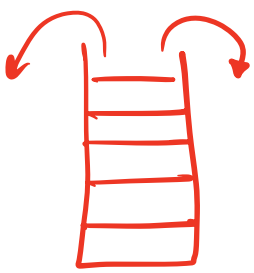
```
void solve (N)
{
    if(N == 0)
        return
    print(N)
    solve (N-1)
}
```

① solve(3)

o/p : 3, 2, 1

② solve(-3)

↳ stackoverflow error



h.w :-

→ ①

```
void solve(N)
{
    if(N==0)
        return

    Solve(N-1)
    print(N)
}
```

O/P for Solve(3)

↳ 5 times

* **Tower of Hanoi**

→ There are N disks of diff sizes, placed on tower A.

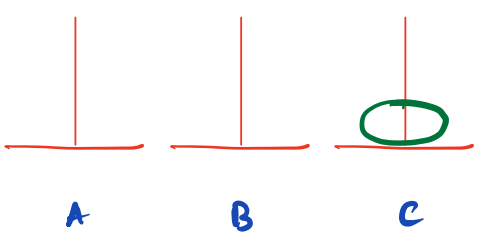
Goal → Move all disks from Tower A to C. (using B)

Constraint :
1. Only 1 disk can be moved at a time. ✓
2. Large disk **cannot** be placed on small disk at any step.

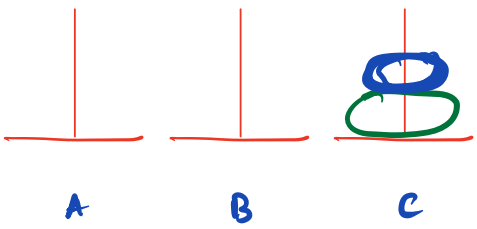
→ Print the moves s.t all the disks move from A to C in **minimum steps**.

N=1 :-

1, A → C



N = 2

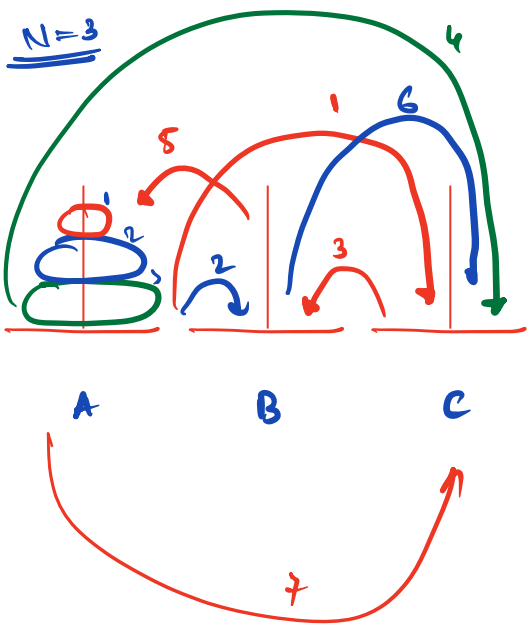


1, A → B

2, A → C

1, B → C

N = 3



1, A → C ✓

2, A → B ✓

1, C → B ✓

3, A → C

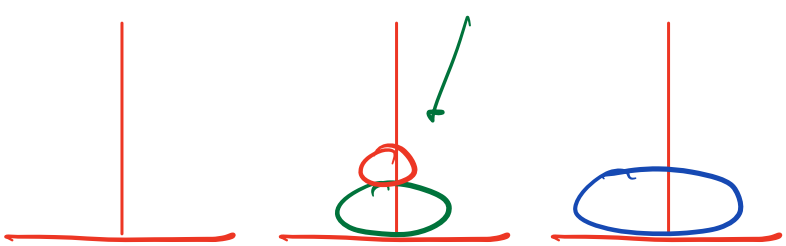
1, B → A

2, B → C

1, A → C

Ans = 7

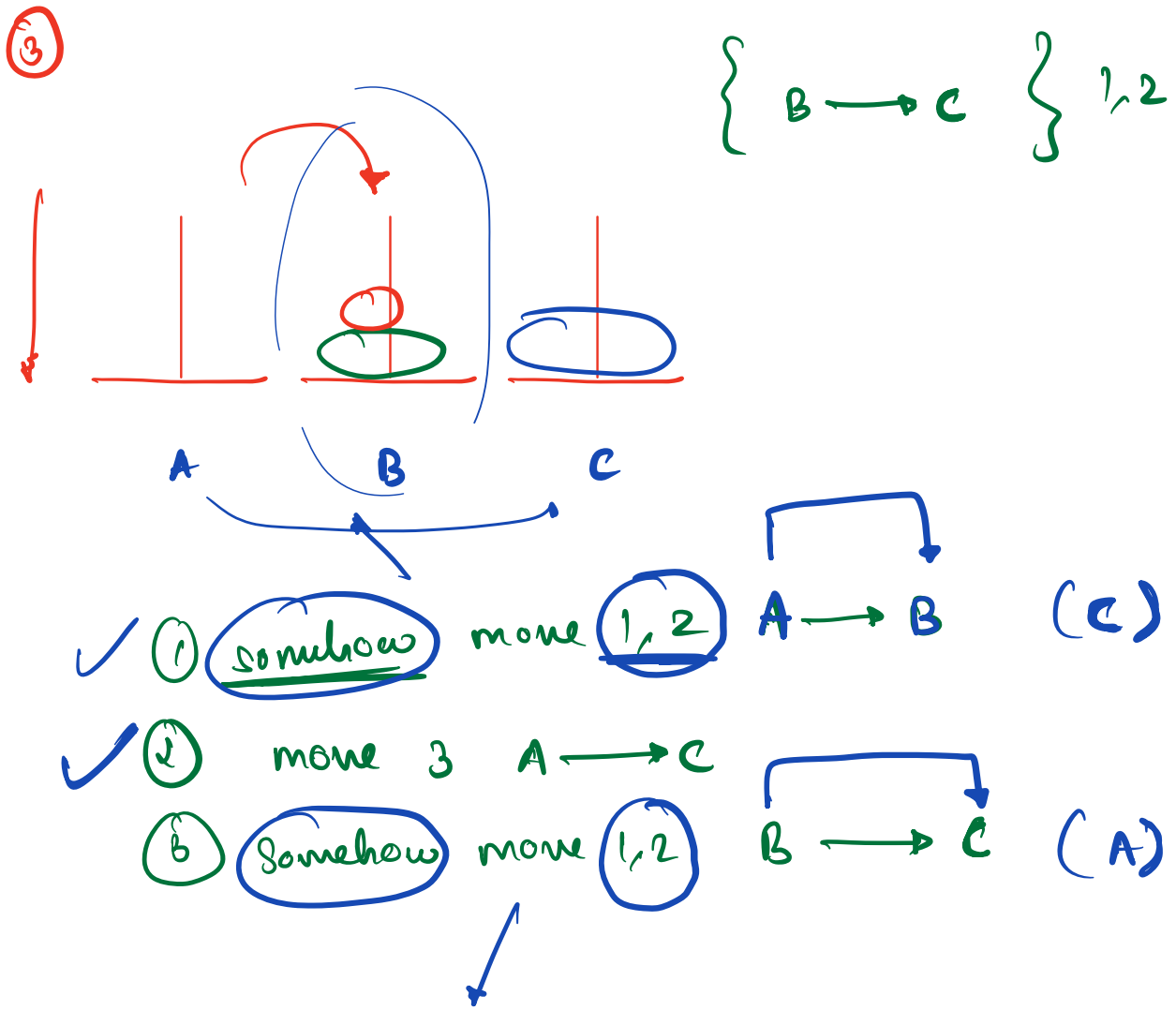
Logic :-



{ S → A, T → B } 1,2

↳ C is intermediate

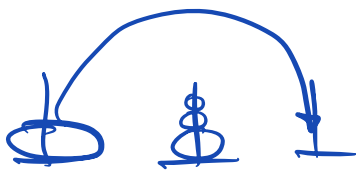
3, A → B



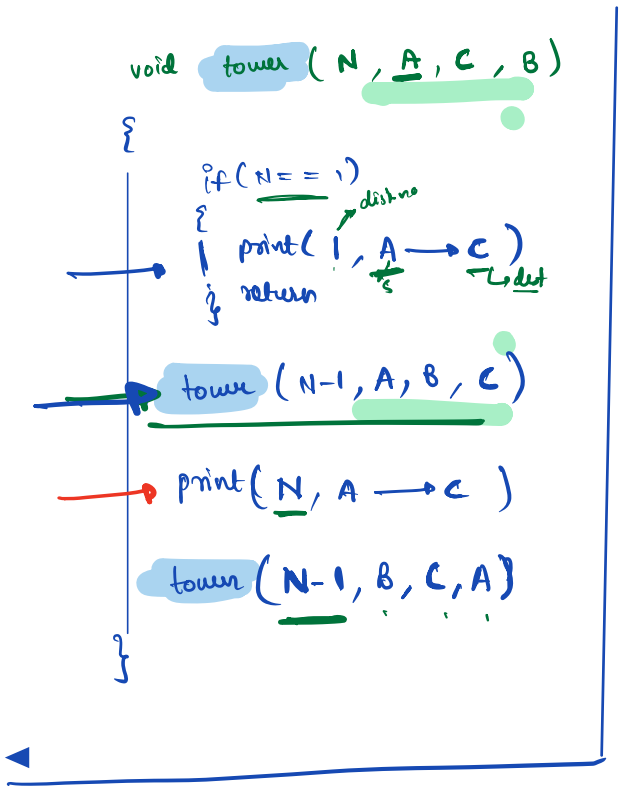
Assumption :- moves N disks from A to C using B

```
void tower(N, A, C, B)
{
    if(N==1)
    {
        print(1, A → C)
        return
    }
    tower(N-1, A, B, C)
    print(N, A → C)
    tower(N-1, B, C, A)
}
```

tower(N, Source, dest, supporting)



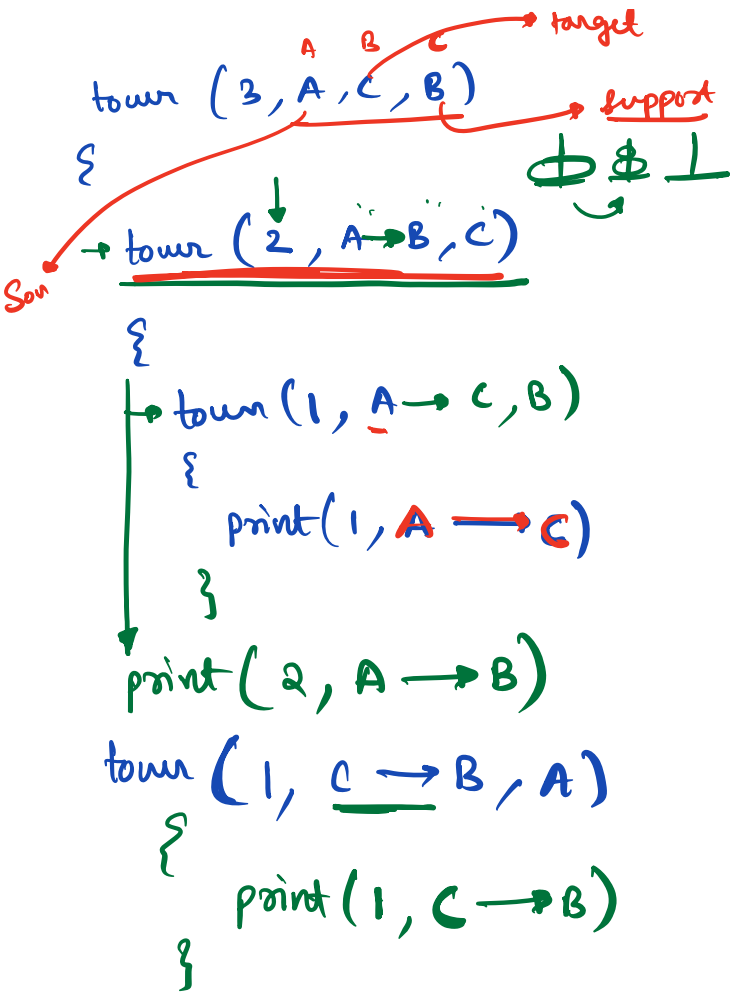
10:18 → 10:25

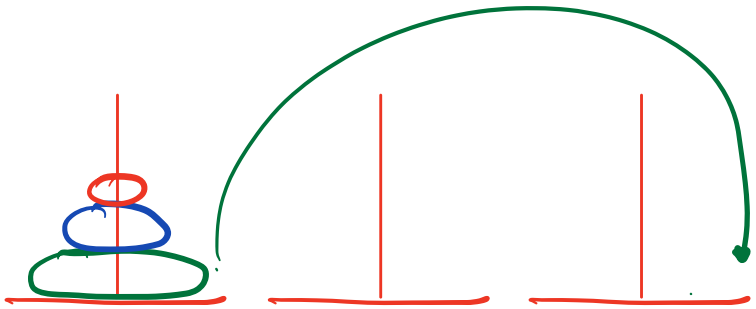


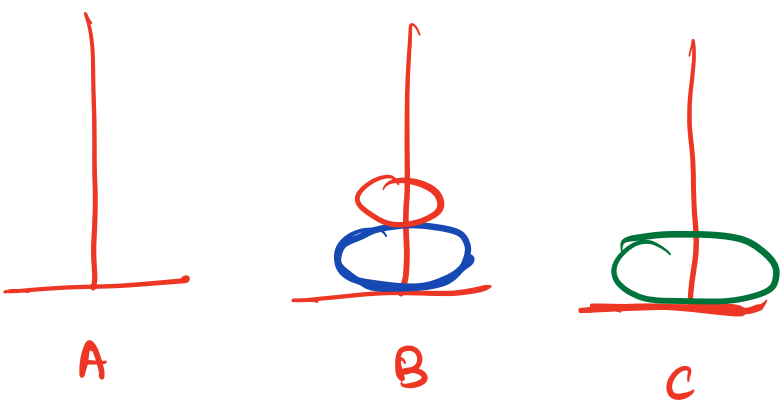
① 1, A → C

② 2, A → B

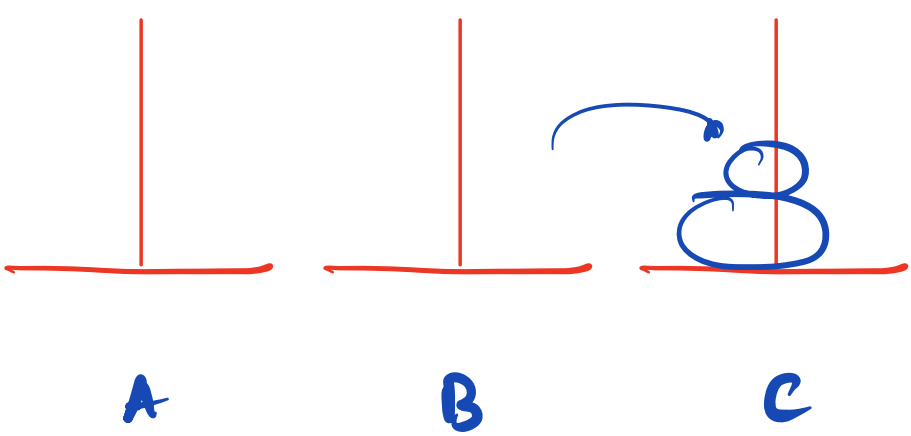
③ 1, C → B



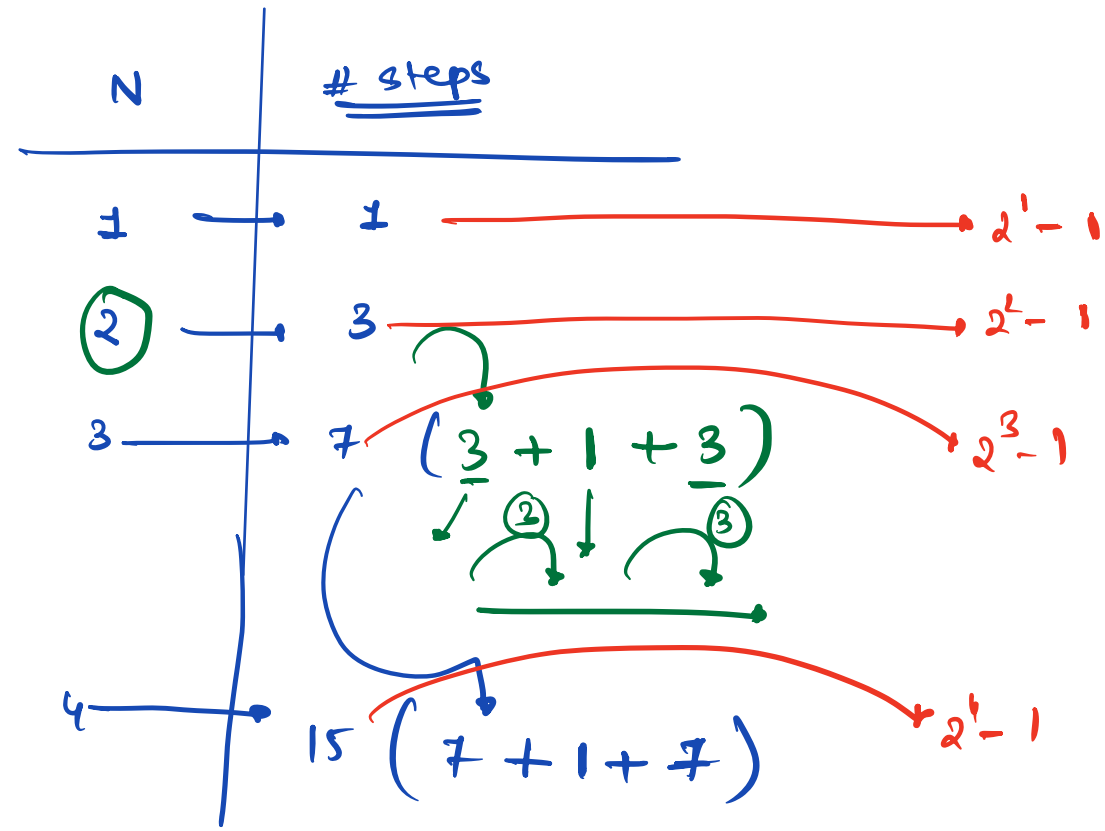




Move 2 disk B → C via A



TC :-



N disks ⟶ # steps $2^N - 1$ steps

TC ⟶ $O(2^N)$

H.W ⟵ SC ⟶ ?

↳ 20 mins

* Recursive Relationship

①

```
int sum(N)
{
    if(N==1)
        return 1
    return N + sum(N-1)
}
```

$$T(N) = T(N-1) + O(1)$$

②

```
void solve(N)
{
    if (N<=1)
        return
    for i → 1 to N
        print(i)
    solve(N/2)
}
```

$$T(N) = T(N/2) + O(N)$$

* Master Theorem

$$T(N) = a * T(N/b) + O(N^d)$$

$a >= 1$ $\quad$ $b > 1$ $\quad$ $d >= 0$

1. $b^d > a \longrightarrow$ TC: $O(N^d)$
2. $b^d = a \longrightarrow$ TC: $O(N^d \log_b(N))$
3. $b^d < a \longrightarrow$ TC: $O(N^{\log_b(a)})$

$$T(N) = T(N/2) + O(N)$$

$T(N) = a * T(N/b) + O(N^d)$

$a >= 1$, $b > 1$, $d >= 0$

$a = 1$

$b = 2$

$d = 1$

① $b^d > a$ → TC: $O(N^d)$

② $b^d = a$ → TC: $O(N^d \log_b(N))$

③ $b^d < a$ → TC: $O(N^{\log_b(a)})$

$b^d = (2)^1 = 2$

Case 1 → $a = 1$

$b^d > a$

TC: $O(N^d) \rightarrow O(N^1) \rightarrow O(N)$

$$T(N) = 1 * T(N-1) + O(1)$$

$a = 1$

$b = 1$

↳ Not applicable for Master Theorem

Q3 :-

$$T(N) = 3T(N/3) + O(N^3)$$ ✓

$a = 3$
$b = 3$
$d = 3$

b^d | a

$3^3 > 3$

TC: $O(N^3)$

HW : $T(N) = 2 * T(N/2) + O(1)$

$a = 2$
$b = 2$
$d = 0$

N^1 $N^0 \rightarrow 1$

b^d | a

$2^0 < 2$

Case 3

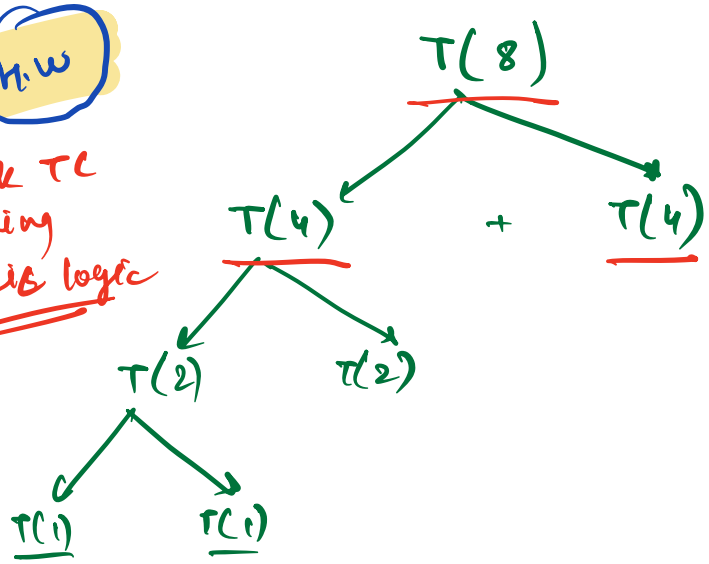
TC: $O(N^{\log_b a})$

$O(N^{\log_2 2})$

TC: $O(N)$

H.W

check TC using basic logic



→ ~ 10-15 min